How to go from an algorithm to a DFA

0110

```
CONTAINS11(w[1..n]):
  found ← FALSE
  for i ← 1 to n
      if i = 1
          last2 ← w[1]
      else
          last2 ← w[i−1]·w[i]
      if last2 = 11
          found ← TRUE
  return found
```

found ∈ {FALSE, TRUE}

what possibilities for last2? state machine

last2 ∈ {ε, 0, 1, 00, 01, 10, 11}

$Q, S, \cancel{\Sigma}, A, \delta$

Q = found × last2

(TRUE, 0)

(FALSE, 01) — 0 → (FALSE, 10); 1 → (TRUE, 11)

ask them

S = (FALSE, ε)

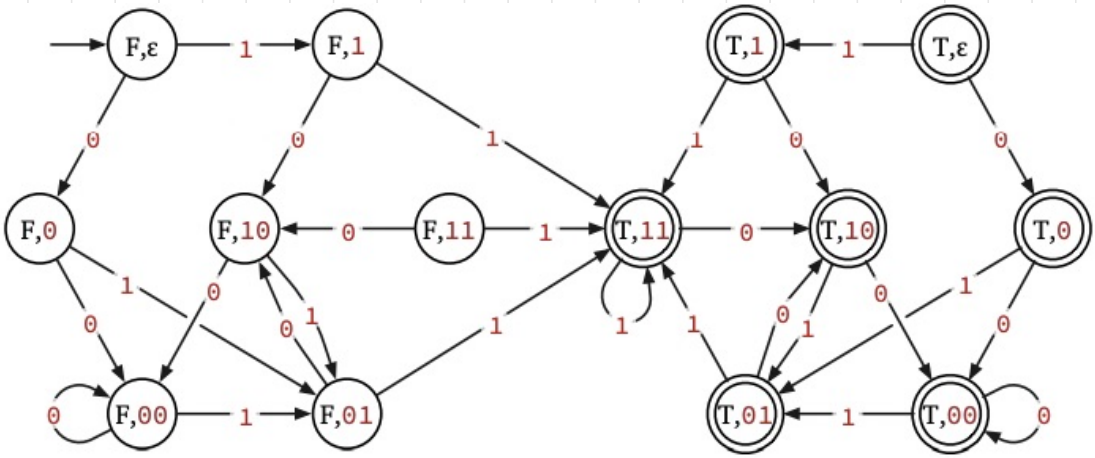
ask them

A = any state w/ TRUE in 1st position

= {(p, q) ∈ Q : p = TRUE}

eg from examples above

δ = as in alg


F,ε
F,1
T,1
T,ε
F,0
F,10
F,11
T,11
T,10
T,0
F,00
F,01
T,01
T,00

Product Construction

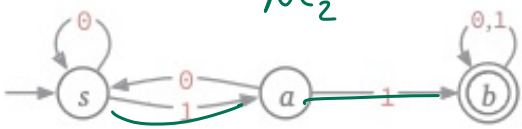
Make a DFA that accepts strings containing both substrings 00 and 11

simulate running both

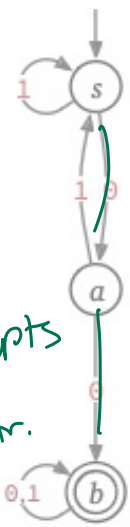
ask them how!

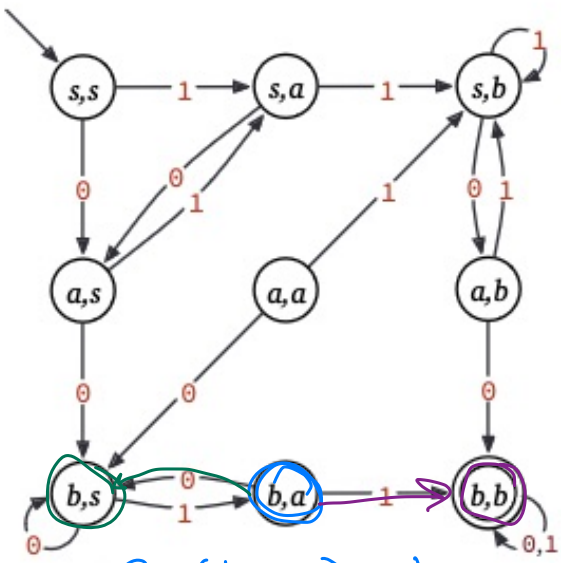
0101011001

M_2 accepts 11 substr.



M_1 accepts 00 substr.





reveal! again, trace of ordered pairs

transitions

eg (b,a)

$\delta_1(b,0) = b$

$\delta_1(b,1) = b$

$\delta_2(a,0) = s$

$\delta_2(a,1) = b$

$M_1 = (Q_1, s_1, A_1, \delta_1)$

$M_2 = (Q_2, s_2, A_2, \delta_2)$

M = machine accepting all strings accepted by both M_1 and M_2

if time let them fill in s, A, δ

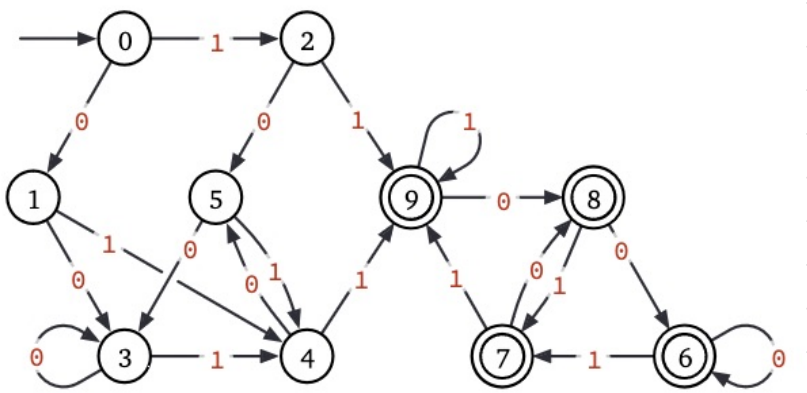
$Q = Q_1 \times Q_2 = ((p,q) : p \in Q_1 \text{ and } q \in Q_2)$

$s = (s_1, s_2)$

$A = ((p,q) : p \in A_1 \text{ and } q \in A_2)$

$\delta((p,q), a) = (\delta_1(p,a), \delta_2(q,a))$

Other languages we can easily build from M_1, M_2?

Another DFA to accept strings w/ 11

Can we simplify?